Experiment No. 8

Application of Optical Flow for Real-Time Object Tracking
and Motion Analysis

Aim

To implement an optical flow-based motion analysis system for real-time object tracking and evaluate its effectiveness in detecting and tracking moving objects in video sequences using Python and OpenCV.

Purpose

While optical flow estimates the motion between consecutive video frames, its real potential lies in applications such as object tracking, traffic monitoring, human activity recognition, autonomous navigation, video surveillance, and sports analytics. By analyzing motion vectors, computer vision systems can continuously track moving objects without relying solely on object detection algorithms.

In this experiment, a real-time object tracking application is developed using optical flow, and object trajectories, motion direction, and movement patterns are analyzed.

Prerequisites

- Basic knowledge of Digital Image Processing
- Familiarity with Python programming
- Python libraries: OpenCV (cv2), NumPy, and Matplotlib
- Understanding of Optical Flow algorithms (Lucas-Kanade and Farneback)
- Jupyter Notebook, Google Colab, or Visual Studio Code

Steps (Procedure)

1. Import the required libraries and load a real-world video or capture live video from a webcam.
2. Identify a moving object or select feature points using Shi-Tomasi corner detection.
3. Apply Lucas-Kanade optical flow to track the object across consecutive frames.
4. Draw motion trajectories and displacement vectors.
5. Calculate the direction and magnitude of motion and display the results in real time.
6. Repeat using Farneback dense optical flow and compare tracking performance.
7. Evaluate the impact of object speed, illumination changes, occlusion, and camera movement.
8. Analyze the tracked motion to estimate displacement, trajectory, and average speed.
9. Compare optical flow tracking with conventional frame-by-frame object detection.
10. Document the observations and discuss practical applications.

Key Concepts and Formulas

- **Tracking pipeline (Lucas-Kanade):** detect Shi-Tomasi corners on the object, track them from frame to frame with pyramidal Lucas-Kanade, discard unreliable points (forward-backward error, status flag, motion outliers), and take the centroid of the surviving points as the object position.
- **Forward-backward check:** track a point from frame t to t+1 and back; if it does not return to its starting position (error above 1 px), the track is unreliable.

- **Tracking pipeline (Farneback):** compute dense flow, subtract the median flow to cancel camera motion, threshold the magnitude, and take the largest moving blob as the object.
- **Displacement:** d = p_{end} - p_{start}; **path length** = sum of $||p_{t+1} - p_t||$.
- **Speed:** v = path length / number of frames (px/frame), and v x fps in px/s. **Direction:** theta = atan2(dy, dx), with the y axis pointing down in image coordinates.
- **Conventional detection baseline:** Gaussian-mixture background subtraction (MOG2), taking the largest foreground blob in every frame.

Observations

- On the synthetic video (object moving about 5 px per frame on a sine path), Lucas-Kanade tracked all 60 frames and recovered the net displacement (293 px horizontally against the true 295 px) at about 1.7 ms per frame. Farneback took about 32 ms per frame, and MOG2 detection about 3 ms.
- Average speed was computed from the track (about 6 px/frame, or 180 px/s at 30 fps, including the vertical component); the direction was about 5 degrees below the horizontal.
- Tracking error against ground truth (mean px, share of frames with an estimate), measured on the synthetic video: baseline - Lucas-Kanade 3.4 px (100%), Farneback 2.3 px (98%), MOG2 7.6 px (92%).
- Illumination change (scene gradually darkening): Lucas-Kanade 2.7 px (100%) and Farneback 2.3 px (98%) were barely affected here, while MOG2 detection found the object in only 20% of frames, because the changing brightness upsets the background model.
- Occlusion (a plain pole hides the object for several frames): Lucas-Kanade lost the points and, with no re-detection step, never recovered (estimate in only 40% of frames); Farneback recovered when the object reappeared (87% of frames, 7.4 px error); MOG2 reached 75% with 14.6 px error.
- Camera motion (the background pans): Lucas-Kanade on the object's own corners was unaffected (1.6 px), Farneback worked after subtracting the median (global) flow (3.4 px), but background subtraction failed completely because every pixel changes.
- Results depend on texture: the synthetic object is strongly textured, so real video with low contrast, blur, or reflections will give larger errors. Check these observations against your own run.

Source Code

File: experiment8_flow_tracking.py

```
"""
Experiment 8: Optical-Flow-based Object Tracking and Motion Analysis
Install:  pip install opencv-python numpy matplotlib
Usage:
  python experiment8_flow_tracking.py                      -> synthetic video with known ground truth
  python experiment8_flow_tracking.py video.mp4            -> your own video (object auto-detected by motion)
  python experiment8_flow_tracking.py video.mp4 --select   -> draw the object box yourself (needs a display)
  python experiment8_flow_tracking.py 0 --show             -> webcam, live display (press q to quit)
"""
import os
import sys
import time
import cv2
import numpy as np
import matplotlib.pyplot as plt

os.makedirs("output", exist_ok=True)
H, W, N, S = 300, 400, 60, 60          # frame size, number of frames, object size
FPS = 30.0
SHOW = "--show" in sys.argv
SELECT = "--select" in sys.argv
args = [a for a in sys.argv[1:] if not a.startswith("--")]


# ---------------------------------------------------------------
# Synthetic video with known object position (ground truth)
# ---------------------------------------------------------------
def gt_center(t):
    """True centre of the object at frame t (moves right along a sine path)."""
    return np.array([40 + 5 * t + S / 2, 120 + 40 * np.sin(0.12 * t) + S / 2])


def make_video(scenario="baseline", seed=7):
    rng = np.random.default_rng(seed)
    bg_big = cv2.GaussianBlur(rng.integers(0, 255, (H + 140, W + 140), dtype=np.uint8), (5, 5), 0)
    obj = cv2.GaussianBlur(rng.integers(40, 255, (S, S), dtype=np.uint8), (3, 3), 0)
    frames = []
    for t in range(N):
        cx, cy = (int(2 * t), int(1 * t)) if scenario == "camera" else (0, 0)   # camera pans
        f = bg_big[cy:cy + H, cx:cx + W].copy()
        x, y = (gt_center(t) - S / 2).astype(int)
        f[y:y + S, x:x + S] = obj
        if scenario == "occlusion":
            f[:, 180:270] = 110                        # plain pole in front of the object
        f = f.astype(np.float32)
        if scenario == "illumination":
            f *= 1.0 - 0.5 * t / N                     # scene gets steadily darker
        f += rng.normal(0, 3, f.shape)                 # sensor noise
        frames.append(cv2.cvtColor(np.clip(f, 0, 255).astype(np.uint8), cv2.COLOR_GRAY2BGR))
    return frames


# ---------------------------------------------------------------
# Step 1: video source
# ---------------------------------------------------------------
real = bool(args)
if real:
    cap = cv2.VideoCapture(0 if args[0] == "0" else args[0])
    FPS = cap.get(cv2.CAP_PROP_FPS) or 30.0
    frames = []
    while len(frames) < 400:
        ok, fr = cap.read()
        if not ok:
            break
        frames.append(cv2.resize(fr, (640, int(fr.shape[0] * 640 / fr.shape[1]))))
    cap.release()
    if len(frames) < 3:
        sys.exit("Could not read enough frames from the video source.")
else:
    print("No video given - using a synthetic video (object moves right on a sine path).")
    frames = make_video("baseline")
grays = [cv2.cvtColor(f, cv2.COLOR_BGR2GRAY) for f in frames]


# ---------------------------------------------------------------
# Step 2: find the object (largest moving region) or let the user select it
# ---------------------------------------------------------------
def largest_blob_box(mask, min_area=150):
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, np.ones((3, 3), np.uint8))
```

```
    mask = cv2.dilate(mask, np.ones((7, 7), np.uint8), iterations=2)
    cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    cnts = [c for c in cnts if cv2.contourArea(c) >= min_area]
    return cv2.boundingRect(max(cnts, key=cv2.contourArea)) if cnts else None


def initial_box(g):
    if real and SELECT:
        x, y, w, h = cv2.selectROI("Select object, press ENTER", frames[0], False)
        cv2.destroyAllWindows()
        return (int(x), int(y), int(w), int(h))
    d = cv2.absdiff(cv2.GaussianBlur(g[0], (5, 5), 0), cv2.GaussianBlur(g[2], (5, 5), 0))
    return largest_blob_box((d > 25).astype(np.uint8) * 255)


# ---------------------------------------------------------------
# Steps 3-5: Lucas-Kanade tracking of Shi-Tomasi corners
# ---------------------------------------------------------------
LK = dict(winSize=(21, 21), maxLevel=3,
          criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 30, 0.01))


def track_lk(g, box, vis_frames=None):
    """Return the per-frame centroid of the tracked points (NaN once the object is lost)."""
    x, y, w, h = box
    mask = np.zeros_like(g[0]); mask[y:y + h, x:x + w] = 255
    p = cv2.goodFeaturesToTrack(g[0], maxCorners=40, qualityLevel=0.1, minDistance=5,
                                blockSize=7, mask=mask)                    # Shi-Tomasi corners on the object
    if not (real and SELECT):
        # keep only corners that actually move between the first two frames (drops static background corners)
        q, st0, _ = cv2.calcOpticalFlowPyrLK(g[0], g[1], p, None, **LK)
        moving = (st0.ravel() == 1) & (np.linalg.norm(q - p, axis=2).ravel() > 1.0)
        if moving.sum() >= 3:
            p = p[moving].reshape(-1, 1, 2)
    centers = np.full((len(g), 2), np.nan)
    centers[0] = p.reshape(-1, 2).mean(axis=0)
    trail = np.zeros_like(frames[0])
    for i in range(1, len(g)):
        if p is None or len(p) < 3:
            break                                                          # object lost
        p1, st, _ = cv2.calcOpticalFlowPyrLK(g[i - 1], g[i], p, None, **LK)
        pb, stb, _ = cv2.calcOpticalFlowPyrLK(g[i], g[i - 1], p1, None, **LK)
        ok = (st.ravel() == 1) & (stb.ravel() == 1) & (np.linalg.norm(p - pb, axis=2).ravel() < 1.0)
        if ok.sum() < 3:
            break
        m = (p1 - p).reshape(-1, 2)
        ok &= np.linalg.norm(m - np.median(m[ok], axis=0), axis=1) < 3.0   # reject points moving differently
        if ok.sum() < 3:
            break
        for a, b in zip(p[ok].reshape(-1, 2), p1[ok].reshape(-1, 2)):
            cv2.line(trail, tuple(map(int, a)), tuple(map(int, b)), (0, 255, 0), 1)
        p = p1[ok].reshape(-1, 1, 2)
        centers[i] = p.reshape(-1, 2).mean(axis=0)
        if vis_frames is not None:
            vis_frames.append((i, p.copy(), trail.copy()))
    return centers


# ---------------------------------------------------------------
# Step 6: Farneback dense-flow tracking (camera motion removed with the median flow)
# ---------------------------------------------------------------
def track_farneback(g, compensate=True, thr=1.5):
    centers = np.full((len(g), 2), np.nan)
    for i in range(1, len(g)):
        flow = cv2.calcOpticalFlowFarneback(g[i - 1], g[i], None, 0.5, 3, 15, 3, 5, 1.2, 0)
        if compensate:
            flow = flow - np.median(flow.reshape(-1, 2), axis=0)           # remove global (camera) motion
        mag = np.linalg.norm(flow, axis=2)
        box = largest_blob_box((mag > thr).astype(np.uint8) * 255)
        if box:
            centers[i - 1] = (box[0] + box[2] / 2, box[1] + box[3] / 2)    # position in frame i-1
    return centers


# ---------------------------------------------------------------
# Step 9: conventional detection baseline (background subtraction, frame by frame)
# ---------------------------------------------------------------
def track_mog2(g):
    mog = cv2.createBackgroundSubtractorMOG2(history=30, varThreshold=25, detectShadows=False)
    centers = np.full((len(g), 2), np.nan)
    for i, im in enumerate(g):
        fg = mog.apply(im)
        if i < 5:
            continue                                                       # let the model learn
        box = largest_blob_box(fg, min_area=300)
```

```
        if box:
            centers[i] = (box[0] + box[2] / 2, box[1] + box[3] / 2)
    return centers


def timed(fn, *a):
    t0 = time.perf_counter(); out = fn(*a)
    return out, (time.perf_counter() - t0) * 1000 / len(a[0])            # ms per frame


# ---------------------------------------------------------------
# Run tracking on the chosen video
# ---------------------------------------------------------------
box = initial_box(grays)
print(f"Initial object box (x, y, w, h): {box}")
vis_log = []
t0 = time.perf_counter()
c_lk = track_lk(grays, box, vis_log)
t_lk = (time.perf_counter() - t0) * 1000 / len(grays)
c_fb, t_fb = timed(track_farneback, grays)
c_bg, t_bg = timed(track_mog2, grays)

# ---------------------------------------------------------------
# Step 8: motion analysis from the Lucas-Kanade track
# ---------------------------------------------------------------
valid = ~np.isnan(c_lk[:, 0])
tr = c_lk[valid]
steps = np.diff(tr, axis=0)
dist = np.linalg.norm(steps, axis=1)
disp = tr[-1] - tr[0]
print("\n---- Motion analysis (Lucas-Kanade track) ----")
print(f"Frames tracked        : {valid.sum()} of {len(grays)}")
print(f"Net displacement      : ({disp[0]:.1f}, {disp[1]:.1f}) px  ->  {np.linalg.norm(disp):.1f} px")
print(f"Path length           : {dist.sum():.1f} px")
print(f"Average speed         : {dist.mean():.2f} px/frame = {dist.mean() * FPS:.1f} px/s (at {FPS:.0f} fps)")
print(f"Mean direction        : {np.degrees(np.arctan2(disp[1], disp[0])):.1f} deg (0 = right, +90 = down)")
print(f"Magnitude/direction of last step: {dist[-1]:.2f} px, {np.degrees(np.arctan2(steps[-1][1], steps[-1][0])):.1f} deg")
print(f"\nTime per frame: Lucas-Kanade {t_lk:.2f} ms | Farneback {t_fb:.2f} ms | MOG2 detection {t_bg:.2f} ms")

# ---------------------------------------------------------------
# Step 7: effect of speed-independent conditions on accuracy (synthetic only)
# ---------------------------------------------------------------
results = {}
if not real:
    gt = np.array([gt_center(t) for t in range(N)])

    def err(c, relative=False):
        """Mean error in px. LK tracks a cloud of corners (not the box centre), so for LK the
        displacement since the first frame is compared instead of the absolute position."""
        if relative:
            k = np.where(np.isfinite(c[:, 0]))[0][0]
            e = np.linalg.norm((c - c[k]) - (gt - gt[k]), axis=1)
        else:
            e = np.linalg.norm(c - gt, axis=1)
        return (np.nanmean(e) if np.isfinite(e).any() else np.nan), 100 * np.isfinite(e).mean()

    print("\n---- Tracking error vs ground truth (mean px, % of frames with an estimate) ----")
    print("(Lucas-Kanade starts from the true object box, as if selected by the user)")
    print(f"{'Scenario':<14}{'Lucas-Kanade':>20}{'Farneback':>20}{'MOG2 detection':>20}")
    for sc in ("baseline", "illumination", "occlusion", "camera"):
        fr = make_video(sc)
        g = [cv2.cvtColor(f, cv2.COLOR_BGR2GRAY) for f in fr]
        x0, y0 = (gt_center(0) - S / 2).astype(int)
        b = (int(x0), int(y0), S, S)      # object selected by the user (frame differencing fails under camera motion)
        row = {}
        row["Lucas-Kanade"] = err(track_lk(g, b), relative=True)
        row["Farneback"] = err(track_farneback(g))
        row["MOG2"] = err(track_mog2(g))
        results[sc] = row
        fmt = lambda r: f"{r[0]:6.2f} px ({r[1]:3.0f}%)"
        print(f"{sc:<14}{fmt(row['Lucas-Kanade']):>20}{fmt(row['Farneback']):>20}{fmt(row['MOG2']):>20}")

# ---------------------------------------------------------------
# Display: trajectories, arrows, error chart
# ---------------------------------------------------------------
rgb = lambda x: cv2.cvtColor(x, cv2.COLOR_BGR2RGB)
last_i, last_p, last_trail = vis_log[-1]
final = cv2.add(frames[last_i].copy(), last_trail)
for pt in last_p.reshape(-1, 2):
    cv2.circle(final, tuple(map(int, pt)), 3, (0, 255, 255), -1)
cv2.polylines(final, [np.int32(tr).reshape(-1, 1, 2)], False, (0, 0, 255), 2)       # centroid trajectory
cv2.arrowedLine(final, tuple(map(int, tr[0])), tuple(map(int, tr[-1])), (255, 0, 0), 2, tipLength=0.05)

fig, ax = plt.subplots(1, 3, figsize=(16, 4.8))
ax[0].imshow(rgb(final)); ax[0].set_title("LK tracking: points (yellow), trajectory (red), net displacement (blue)")
```

```
ax[0].axis("off")
ax[1].plot(*tr.T, "r-o", ms=2, label="Lucas-Kanade")
for c, lab, col in [(c_fb, "Farneback", "g"), (c_bg, "MOG2 detection", "b")]:
    m = ~np.isnan(c[:, 0]); ax[1].plot(*c[m].T, col + ".", ms=3, label=lab)
if not real:
    ax[1].plot(*gt.T, "k--", label="Ground truth")
ax[1].invert_yaxis(); ax[1].set_title("Trajectories (pixel coordinates)"); ax[1].legend(fontsize=8)
ax[1].set_aspect("equal")
if results:
    labs = list(results); x = np.arange(len(labs)); wd = 0.27
    for k, (m, col) in enumerate([("Lucas-Kanade", "r"), ("Farneback", "g"), ("MOG2", "b")]):
        ax[2].bar(x + (k - 1) * wd, [np.nan_to_num(results[s][m][0], nan=0) for s in labs], wd, color=col, label=m)
    ax[2].set_xticks(x); ax[2].set_xticklabels(labs); ax[2].set_ylabel("mean error (px)")
    ax[2].set_title("Tracking error by condition"); ax[2].legend(fontsize=8)
else:
    ax[2].plot(dist, "r"); ax[2].set_title("LK speed per frame (px)"); ax[2].set_xlabel("frame")
plt.tight_layout(); plt.savefig("output/exp8_tracking.png", dpi=150); plt.show()

# Optional live view of the Lucas-Kanade tracker
if SHOW:
    for i, p, trail in vis_log:
        v = cv2.add(frames[i].copy(), trail)
        for pt in p.reshape(-1, 2):
            cv2.circle(v, tuple(map(int, pt)), 3, (0, 255, 255), -1)
        cv2.imshow("Optical flow tracking (q to quit)", v)
        if cv2.waitKey(int(1000 / FPS)) & 0xFF == ord("q"):
            break
    cv2.destroyAllWindows()
```

Questions and Answers

1. How does object tracking differ from object detection?

Aspect	Object Detection	Object Tracking
Goal	Find and classify objects in a single image or frame	Follow the same object across frames and keep its identity
Input	One frame (no temporal information needed)	A sequence of frames, with the object's previous state
Output	Bounding boxes and class labels for each frame	A trajectory (position over time) with a consistent ID
Computation	Usually heavy, run on every frame	Lighter after initialization; uses motion and appearance continuity
Examples	YOLO, SSD, Faster R-CNN, Haar cascade	Lucas-Kanade tracking, KCF, SORT, DeepSORT

2. Explain how Optical Flow can be used for real-time object tracking.

Optical flow gives the motion of image points between consecutive frames, so an object can be followed by moving its position along the estimated flow.

1. Select the object (by user selection or motion detection) and detect strong corners on it (Shi-Tomasi).
2. For every new frame, estimate where each point has moved with pyramidal Lucas-Kanade.
3. Discard points that fail the forward-backward or status checks, and update the object position from the remaining points (e.g., the centroid or median motion).
4. Draw trajectories and compute displacement, direction, and speed; re-detect points if too many are lost.
5. Because it only needs to compute motion for a few points, it runs in real time (about 2 ms per frame in this experiment), and no detector is needed on every frame.

3. What is the role of Shi-Tomasi Corner Detection in the Lucas-Kanade Optical Flow algorithm?

Lucas-Kanade solves for motion using the structure matrix A^TA of image gradients in a window. This system is solvable and stable only if both eigenvalues of A^TA are large, which happens at corners. On flat areas both eigenvalues are small (no motion information), and on edges only one is large (aperture problem: motion can be measured only perpendicular to the edge). The Shi-Tomasi detector selects points where the smaller eigenvalue min(lambda_1, lambda_2) is above a threshold, so those are exactly the points that Lucas-Kanade can track reliably. In OpenCV this is `cv2.goodFeaturesToTrack`.

4. Why is Optical Flow suitable for motion analysis in videos?

- It measures motion directly from the pixel data, giving both direction and magnitude of movement
- It needs no object model, training data, or class labels, so it can analyze any moving thing
- It gives motion at point or pixel level, allowing both tracking and segmentation of moving regions
- It is efficient (especially sparse flow) and works in real time
- It works with moving cameras too, because camera motion can be estimated and separated from object motion
- Quantities such as displacement, trajectory, speed, and direction can be computed from it

5. What challenges arise while tracking fast-moving or partially occluded objects?

- **Fast motion:** optical flow assumes small displacement; when the object moves farther than the search window, the estimate fails (in Experiment 7 the error without an image pyramid grew to equal the full displacement). Image pyramids, higher frame rates, or predictive motion models (Kalman filter) help. Motion blur also removes the texture needed for tracking.

- **Occlusion:** tracked points disappear or jump onto the occluding object; points are lost and tracking drifts or fails. Remedies: forward-backward checks, re-detecting corners, motion prediction, and re-identification after the object reappears.
- Appearance and illumination change, and objects leaving the frame, add further difficulty

6. Compare Optical Flow-based tracking with deep learning-based object tracking methods.

Aspect	Optical Flow Tracking	Deep Learning Tracking (e.g., SORT/DeepSORT, Siamese trackers, transformers)
Basis	Low-level motion and brightness consistency	Learned appearance and motion features
Training data	None	Large labeled datasets
Speed and hardware	Very fast; CPU is enough	Heavier; often needs GPU
Robustness	Sensitive to occlusion, lighting change, blur, fast motion; drifts over time	More robust to appearance change and occlusion; can re-identify objects
Object knowledge	Does not know what the object is	Knows object classes and identities
Best for	Short-term tracking, motion analysis, embedded systems	Long-term, multi-object tracking in complex scenes

7. How can Optical Flow be used in traffic monitoring and autonomous driving systems?

- **Traffic monitoring:** detect moving vehicles, estimate their speed and direction, count vehicles, detect wrong-way driving, stopped vehicles, or congestion from the flow field
- **Autonomous driving:** estimate ego-motion (visual odometry), detect and track other vehicles and pedestrians, estimate time to collision from expanding flow, detect obstacles and moving objects, and support lane keeping and parking
- Dense flow helps separate moving objects from the static scene, and flow is a useful input to deep driving models

8. Explain the effect of camera motion on Optical Flow estimation.

Camera motion makes every pixel move, so the flow field contains both ego-motion (global flow from camera translation, rotation, and zoom) and the real motion of objects. As a result, static background appears to move and moving objects are harder to separate. Fast camera motion also causes large displacements and motion blur, which break the small-motion assumption. Typical handling:

- Estimate the dominant (global) motion, for example with the median flow or a homography fitted with RANSAC, and subtract it; objects then appear as regions whose flow differs from the global one (used in this experiment)
- Track the object's own features (Lucas-Kanade), which is unaffected by background motion
- Use image pyramids and stabilization for large motions
- Note that background subtraction fails entirely with a moving camera

9. Mention five real-world applications where motion analysis using Optical Flow is commonly employed.

1. Video surveillance: moving object detection, people tracking, and crowd analysis
2. Autonomous vehicles and ADAS: ego-motion, obstacle, and pedestrian motion
3. Robotics and drones: visual odometry, navigation, obstacle avoidance, and optical-flow sensors for hovering
4. Sports analytics and human activity or gesture recognition
5. Video stabilization, video compression (motion estimation), and frame interpolation; also medical imaging (e.g., heart wall motion) and augmented reality

10. How can Optical Flow improve the performance of surveillance, robotics, and human activity recognition systems?

- **Surveillance:** separates moving objects from the static background without a background model, tracks people and vehicles between detections, reduces the number of detector calls, and flags abnormal movement (running, loitering, wrong direction).
- **Robotics:** provides fast, low-cost motion cues for navigation, visual odometry, obstacle avoidance, and tracking of targets or landmarks, even on small embedded processors.
- **Human activity recognition:** flow fields capture motion patterns (walking, waving, falling) independently of appearance; two-stream networks combine optical flow with RGB frames for higher accuracy than appearance alone.

Conclusion

An optical flow-based tracking and motion analysis system was implemented using Python and OpenCV. Shi-Tomasi corners tracked with pyramidal Lucas-Kanade gave fast and accurate tracking, from which displacement, trajectory, direction, and average speed were computed, while Farneback dense flow was more robust to occlusion at higher computational cost. Compared with frame-by-frame background-subtraction detection, optical flow tracking coped far better with illumination change and camera motion, but needs re-detection or prediction to handle long occlusions, fast motion, and drift.